SOFTWARE REQUIREMENTS SPECIFICATION

Requirements Engineering Assistant (RE Assistant)

Version 5.0
April 18, 2026
University of Hildesheim

Integrated Research Project, Project C2

Prepared by:
Hunain Murtaza

IEEE Std 830-1998, Software Requirements Specifications

Revision History

Version	Description	Author	Date
1.0	Initial draft, Phase 1 elicitation complete	RE Team	2025-09-01
2.0	Phase 2 NFR depth coverage added	RE Team	2025-10-15
3.0	Phase 3 IEEE-830 documentation sections	RE Team	2025-11-20
4.0	Domain gate redesign, SRS Coverage Enricher	RE Team	2026-01-10
5.0	Iteration 8, Project management layer, dual task modes, upload pipeline	RE Team	2026-03-18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Requirements Engineering Assistant (RE Assistant), an AI-powered elicitation and documentation tool developed at the University of Hildesheim as part of Integrated Research Project C2.

This document is written in compliance with IEEE Standard 830-1998. It is intended for the project team, academic supervisors, and future maintainers or evaluators of the system. It serves as the authoritative reference for all development, verification, and evaluation activities.

1.2 Scope

The RE Assistant automates the requirements elicitation process through a structured, three-phase conversational interview with a stakeholder. The system guides users through functional domain coverage, non-functional requirement depth, and IEEE 830 documentation collection, ultimately generating a complete, annotated SRS document.

The product is named RE Assistant (internal codename: Project C2). Its primary objectives are:

- Automate structured stakeholder interviews for requirements elicitation.
- Detects real-time coverage gaps against 14 IEEE 830 categories.
- Enforce completeness across functional domains and 6 mandatory NFR categories.
- Generate a full IEEE 830-1998 SRS document from the elicited requirements.
- Support ablation study evaluation by toggling gap detection on or off per session.

The system is not intended for production deployment and does not support multi-user authentication, persistent session storage across server restarts, or integration with external project management tools.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
RE Assistant	Requirements Engineering Assistant, the software system described in this SRS
SRS	Software Requirements Specification (IEEE Std 830-1998)
FR	Functional Requirement
NFR	Non-Functional Requirement
IEEE 830	IEEE Standard for Software Requirements Specifications (1998 revision)
LLM	Large Language Model, the AI backend used for all generative tasks
Domain Gate	A gating mechanism that tracks the elicitation status of each functional domain
SMART	Specific, Measurable, Achievable, Relevant, Time-bound, quality criteria for requirements
Gap Detector	The subsystem that analyses current requirements against 14 IEEE 830 coverage categories

Term / Acronym	Definition
Phase 1 / fr	Functional requirement elicitation phase, domain-driven
Phase 2 / nfr	Non-functional requirement depth coverage phase
Phase 3 / ieee	IEEE 830 narrative documentation collection phase
SRS-only mode	Task type where domain seeding is skipped and the system moves directly to Phase 3
DXA	Document XML Attribute unit, 1440 DXA = 1 inch (Word/OOXML units)
REST API	Representational State Transfer Application Programming Interface
Flask	Python micro web framework used for the HTTP server
JSON	JavaScript Object Notation, data format used for project and log persistence
Probe	An actively generated question targeting an under-covered domain
Decomposition	Automated generation of implied requirements from domain templates

1.4 References

1. IEEE Computer Society. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE, 1998.
2. Wiegers, K. E., & Beatty, J. Software Requirements (3rd ed.). Microsoft Press, 2013.
3. Project C2 SLR Planning Document, Version 2.0, University of Hildesheim, 2025.
4. Project C2 SLR Synthesis Questionnaire, University of Hildesheim, 2025.
5. OpenAI Platform Documentation. https://platform.openai.com/docs (accessed April 2026).
6. Ollama API Documentation. https://genai-01.uni-hildesheim.de/ollama (internal, accessed April 2026).
7. Flask Documentation. https://flask.palletsprojects.com (accessed April 2026).

1.5 Overview

The remainder of this document is organized as follows. Section 2 provides an overall description of the product, including its context, functions, user characteristics, operating environment, and assumptions. Section 3 specifies the detailed functional and non-functional requirements. Appendices A through D provide a requirements traceability matrix, coverage and quality report, conversation transcript summary, and design-derived stub inventory respectively.

2. Overall Description

2.1 Product Perspective

The RE Assistant is a standalone, single-page web application. It is not a component of a larger existing system; rather, it is an independent research prototype developed to investigate the feasibility of automating requirements elicitation through conversational AI.

The system interacts with an external Large Language Model (LLM) backend, either OpenAI's GPT-4o via the OpenAI API, or a locally hosted model via the Ollama REST interface at the University of Hildesheim's GenAI infrastructure. The system does not store or transmit personally identifiable information beyond session metadata.

From a data perspective, the system maintains three types of persistent artefacts on the file system: project records (JSON files in the projects/ directory), session logs (JSON files in the logs/ directory), and generated SRS documents (Markdown files in the output/ directory). All in-memory session state is lost on server restart.

The product perspective can be summarized with the following system boundary: the browser (the UI) communicates with the Flask REST API; the API communicates with the LLM provider and manages all in-memory session state; persistence is achieved through lightweight JSON file I/O.

2.2 Product Functions

The RE Assistant provides the following major functions:

- Conversational Requirements Elicitation: The system conducts a structured stakeholder interview across three phases, functional domain coverage (Phase 1), non-functional requirement depth (Phase 2), and IEEE 830 narrative section collection (Phase 3). Each phase is driven by a dedicated prompt architect context builder.
- Project Management: Users can create, read, update, and delete named projects. Each project persists as a JSON file and links to a session identifier. The UI lists all projects and allows navigation between them.
- Dual Task Modes: The system supports two operating modes. Elicitation mode conducts the full three-phase interview. SRS-only mode accepts a pre-existing requirements file and proceeds directly to Phase 3 for IEEE 830 documentation collection.
- Requirements File Upload and Preprocessing: Users may upload a .txt or .json file of existing requirements. The preprocessing pipeline applies SMART rewriting, type and category classification, and atomic splitting via LLM, then injects the results into the session and seeds the domain gate from discovered labels.
- Dynamic Functional Domain Gate: The system dynamically discovers functional domains from the first user message and subsequent re-seeding passes (at turns 10, 20, and 30). Each domain progresses through three statuses: unprobed, partial, and confirmed. The gate is satisfied when at least 80% of in-scope domains are confirmed and all have been actively probed.
- Real-Time Gap Detection: The GapDetector analyses the current requirements set against 14 IEEE 830 coverage categories and reports missing or thin coverage areas after each conversation turn.
- NFR Depth Enforcement: Six mandatory NFR categories (performance, usability, security/privacy, reliability, maintainability, and compatibility) each require at least two measurable requirements before Phase 3 can begin.

- Template-Aware Requirement Decomposition: Per-domain coverage checklists drive automated gap-targeted requirement generation. Re-decomposition is triggered when a domain grows by three or more requirements since the last pass.
- SMART Quality Annotation and Rewriting: Every extracted requirement is evaluated against SMART criteria. Vague requirements are automatically rewritten by the LLM before being committed to state.
- SRS Document Generation: At session end, the system generates a complete IEEE 830-1998 SRS document in Markdown format, including all seven structural sections, a traceability matrix, a coverage report, a conversation transcript summary, and design-derived stub annotations.
- Session Log Browser and Replay: The UI exposes a log browser that displays all past sessions linked to the current project. Any session can be replayed to review the conversation turns, domain gate state, and NFR coverage snapshots.
- Ablation Study Support: Gap detection can be toggled on or off per session via the UI or the API, enabling controlled evaluation experiments comparing elicitation quality with and without gap feedback.

2.3 User Characteristics

The RE Assistant is intended for the following user classes:

User Class	Description	Technical Level
Requirements Engineer	Primary user; conducts stakeholder interviews using the system	High, familiar with RE concepts, IEEE 830, SMART criteria
Researcher / Evaluator	Uses the system in ablation study mode to compare elicitation quality	High, understands experimental design and requirements quality metrics
Stakeholder / Domain Expert	Provides domain knowledge through the conversational interface; may not have technical background	Low to medium, no knowledge of RE methodology required
System Administrator	Deploys and maintains the server; configures LLM provider via CLI arguments	High, comfortable with Python, CLI tools, environment variables

No specialized knowledge of requirements engineering is required from a stakeholder using the conversational interface. The system is designed to guide any domain expert through the elicitation process using natural language.

2.4 Operating Environment

The RE Assistant operates in the following environment:

- Server: Python 3.10 or higher. Flask web framework. Unix-compatible operating system (Ubuntu 22.04 LTS recommended). Minimum 2 GB RAM; 4 GB recommended for concurrent sessions.
- Client: Any modern web browser supporting ES2020+ and the Fetch API (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+). No browser extensions or plugins are required.

- Network: HTTPS or HTTP access to the Flask server on port 5000 (configurable). Outbound HTTPS access to the OpenAI API (api.openai.com) or to the University of Hildesheim's Ollama instance (genai-01.uni-hildesheim.de).
- LLM Backend: OpenAI GPT-4o (via OPENAI_API_KEY environment variable) or Ollama-hosted model (via OLLAMA_API_KEY and OLLAMA_BASE_URL). A stub provider is available for offline testing without LLM access.
- Storage: Local file system with write access to logs/, output/, and projects/ subdirectories. No database server is required.

2.5 Assumptions and Dependencies

The following assumptions and dependencies apply to this specification:

1. A valid LLM API key (OpenAI or Ollama) is available and configured via environment variables before the server is started.
2. The server is deployed on a machine with reliable outbound internet access to the LLM API endpoint.
3. The file system permits read/write operations on the logs/, output/, and projects/ directories at the application root.
4. A single server process handles all sessions; no load balancing or horizontal scaling is assumed.
5. The stakeholder's first message is sufficiently descriptive to allow the LLM to seed an initial domain list. Vague opening messages are compensated for by re-seeding at turns 10, 20, and 30.
6. The LLM provider consistently emits well-formed REQ and SECTION XML tags as instructed. Malformed output is handled by fallback regex patterns, but may result in missed requirements.
7. Session data is ephemeral; the server operator is responsible for exporting session logs and SRS documents before server restarts.
8. The system is used in a single-user, non-concurrent context. Concurrent access to the same session from multiple browser tabs may cause state inconsistencies.

2.6 User Documentation

The following user documentation artefacts are provided or planned for the RE Assistant:

- README.md: Comprehensive technical documentation covering architecture, installation, API reference, component reference, SRS output format, ablation study support, configuration, and known limitations. Maintained as part of the source repository.
- In-Application UI Guidance: The single-page web interface includes contextual labels, status indicators (domain gate ring, IEEE-830 coverage ring, gap panel), and tooltips to guide users through the elicitation process without requiring prior training.
- Session Log Browser: The built-in log browser allows users to replay and review any past session, serving as an informal audit trail and learning resource.
- API Reference: The README.md documents all REST endpoints with request/response schemas, enabling programmatic use of the system by researchers or integrators.

3. Specific Requirements

3.1 Functional Requirements

This section specifies all functional requirements for the RE Assistant. Requirements are grouped by functional area and presented in tables for traceability. All requirements are stated using the keyword 'shall' in accordance with IEEE 830-1998.

3.1.1 Session Management

ID	Category	Requirement
FR-01	Session Management	The system shall generate a unique 8-character hexadecimal session identifier for each new session initiated via POST /api/session/start.
FR-02	Session Management	The system shall create an in-memory ConversationState object upon session start, containing all runtime states (turns, requirements, NFR coverage, domain gate, phase4 sections).
FR-03	Session Management	The system shall write a session_start event to a JSON log file (logs/session_<id>.json) within 100 ms of session creation.

3.1.2 Project Management

ID	Category	Requirement
FR-04	Project Management	The system shall allow users to create a named project via POST /api/projects/create, persisting the project as a JSON file in the projects/ directory with id, name, description, task_type, created_at, updated_at, session_id, req_count, and srs_path fields.
FR-05	Project Management	The system shall support retrieval, update, and deletion of projects via GET /api/projects/<id>, PUT /api/projects/<id>, and DELETE /api/projects/<id> endpoints respectively.
FR-06	Project Management	The system shall return a lightweight project card list via GET /api/projects, sorted by modification time descending, containing id, name, description, task_type, created_at, updated_at, session_id, and req_count for each project.

3.1.3 Task Modes

ID	Category	Requirement
FR-07	Task Modes	The system shall support two task modes: elicitation (full three-phase interview) and srs_only (direct IEEE-830 documentation collection). The mode shall be specified via the task_type parameter in the session start request.

ID	Category	Requirement
FR-08	Task Modes	In srs_only mode, the system shall skip domain seeding and Phase 1/Phase 2 elicitation and advance directly to Phase 3 (IEEE section collection) from the first conversation turn.

3.1.4 Elicitation

ID	Category	Requirement
FR-09	Elicitation	The system shall process each user message via POST /api/session/chat and return an assistant response generated by the configured LLM provider within the same HTTP response.
FR-10	Elicitation	The system shall construct a phase-aware system prompt for each LLM call using the PromptArchitect component, selecting the appropriate context builder (domain context, NFR context, or IEEE section context) based on the current elicitation phase.
FR-11	Elicitation	The system shall include the most recent 10 conversation turns (configurable via MAX_HISTORY_TURNS) in the LLM context window for each chat call.
FR-12	Elicitation	The system shall extract structured requirements from the assistant response by parsing REQ XML tags using the RequirementExtractor component, with fallback regex patterns for malformed output.
FR-13	Elicitation	The system shall extract IEEE 830 narrative section content from the assistant response by parsing SECTION XML tags and committing identified sections to the session state.
FR-14	Elicitation	The system shall persist each conversation turn to the session log as a turn event containing turn_id, user_message, assistant_message, categories_updated, and gap_report.

3.1.5 Domain Gate

ID	Category	Requirement
FR-15	Domain Gate	The system shall seed the domain gate from the first user message via an LLM call that returns a JSON list of functional domain specifications (label, scope description, and expected requirement types).
FR-16	Domain Gate	The system shall re-seed the domain gate at conversation turns 10, 20, and 30 by sampling up to 40 requirements distributed evenly across categories and invoking the DomainDiscovery.reseed() method.
FR-17	Domain Gate	Each functional domain shall progress through three statuses, unprobed, partial, and confirmed, where confirmed requires at least 3 assigned requirements AND at least 1 active probe (probe_count >= 1).

ID	Category	Requirement
FR-18	Domain Gate	The domain gate shall be considered satisfied when: (a) at least 80% of in-scope (non-excluded) domains are confirmed, AND (b) every in-scope unconfirmed domain has been probed at least once.
FR-19	Domain Gate	The system shall match each extracted requirement to its corresponding functional domain using the DomainDiscovery.match_requirement_to_domain() method, updating domain req_ids and status accordingly.
FR-20	Domain Gate	The system shall compute domain completeness percentage as confirmed_count / active_count * 100, excluding excluded domains from both numerator and denominator.

3.1.6 Requirements

ID	Category	Requirement
FR-21	Requirements	The system shall classify each extracted requirement as functional, non_functional, or constraint based on the req_type field in the REQ tag, defaulting to unknown for unrecognized types.
FR-22	Requirements	The system shall evaluate each extracted requirement against SMART criteria via an LLM batch check and rewrite vague requirements before committing them to session state.
FR-23	Requirements	The system shall classify each non-functional requirement into one of six NFR categories (performance, usability, security_privacy, reliability, maintainability, compatibility) using the DomainDiscovery.classify_nfr() method.
FR-24	Requirements	The system shall deduplicate incoming requirements using a message similarity threshold of 0.7, discarding requirements whose normalized text similarity to any existing requirement exceeds this threshold.
FR-25	Requirements	The system shall assign each requirement a unique identifier with type-prefixed sequential numbering (FR-NNN, NFR-NNN, CON-NNN), a category, a domain key, a source tag (elicited, decomposed, or uploaded), and a creation timestamp.

3.1.7 Decomposition

ID	Category	Requirement
FR-26	Decomposition	The system shall decompose template-derived implied requirements for each domain that has at least 2 assigned requirements and has not yet been decomposed, using per-domain coverage checklists to target uncovered dimensions.
FR-27	Decomposition	The system shall re-decompose a domain when it has grown by 3 or more requirements since the last decomposition pass, up to

ID	Category	Requirement
		a cap of 3 domains per turn for standard systems and 5 domains per turn for complex systems.
FR-28	Decomposition	Decomposed requirements prefixed with [NFR] shall be stored as non-functional requirements with their NFR category classified by DomainDiscovery.classify_nfr().

3.1.8 Phase Transitions

ID	Category	Requirement
FR-29	Phase Transitions	The system shall transition from Phase 1 to Phase 2 when at least 10 functional requirements have been collected AND the domain gate is satisfied.
FR-30	Phase Transitions	The system shall transition from Phase 2 to Phase 3 when each of the 6 mandatory NFR categories has at least 2 measurable requirements (configurable via MIN_NFR_PER_CATEGORY = 2).
FR-31	Phase Transitions	The system shall transition to SRS generation readiness after a maximum of 60 conversation turns regardless of domain gate or NFR coverage state, preventing indefinite sessions.

3.1.9 Gap Detection

ID	Category	Requirement
FR-32	Gap Detection	The system shall analyse the current requirements set against 14 IEEE 830 coverage categories after each conversation turn and return a gap report containing per-category coverage status (not_started, partially_covered, covered) and missing items.
FR-33	Gap Detection	The gap detection subsystem shall be toggleable per session via the gap_detection boolean parameter in the session start request; when disabled, the GapDetector shall return a full-coverage report (100%, no gaps).
FR-34	Gap Detection	The system shall inject a coverage hint into the Phase 1 prompt context when a gap is detected, suggesting a follow-up question targeting the uncovered area without mandating verbatim reproduction of the hint.

3.1.10 SRS Generation

ID	Category	Requirement
FR-35	SRS Generation	The system shall generate a complete IEEE 830-1998 SRS document in Markdown format upon POST /api/session/generate_srs, saving the file as output/SRS_<session_id>_<timestamp>.md.

ID	Category	Requirement
FR-36	SRS Generation	The generated SRS shall include: a header with dual coverage metrics; Section 1 (Introduction) with purpose, scope, definitions, and references; Section 2 (Overall Description) with product perspective, functions, user characteristics, operating environment, and assumptions; Section 3 (Specific Requirements) with all FRs, NFRs, and constraints; Section 3.2 (External Interfaces); Section 3.3–3.6 (performance, database, design constraints, system attributes); and Appendices A–D.
FR-37	SRS Generation	The SRS Coverage Enricher shall fill any empty IEEE 830 sections using a priority chain: Phase 3 answers first, then LLM synthesis from session context, then architect-review stubs tagged with [STUB].
FR-38	SRS Generation	Appendix A of the generated SRS shall contain a requirements traceability matrix linking each requirement ID to its category, domain, source, and turn of elicitation.
FR-39	SRS Generation	Appendix D of the generated SRS shall contain domain-agnostic stub blocks for all unconfirmed domains and uncovered structural sections, tagged [D], with no hard-coded domain-specific content.

3.1.11 Upload Pipeline

ID	Category	Requirement
FR-40	Upload Pipeline	The system shall accept a requirements file (plain text or JSON) via POST /api/session/upload_requirements, parse it into individual requirement lines, and pass each line through the RequirementPreprocessor pipeline.
FR-41	Upload Pipeline	The RequirementPreprocessor shall evaluate each requirement for SMART compliance, rewrite non-compliant requirements, assign req_type (functional / non_functional / constraint) and category_label via a single batched LLM call, and flag whether each requirement was rewritten or split.
FR-42	Upload Pipeline	After preprocessing, the system shall seed the domain gate from the discovered functional category labels using DomainGate.seed_from_labels() without an additional LLM domain-seed call.

3.1.12 Logging

ID	Category	Requirement
FR-43	Logging	The system shall expose GET /api/logs returning metadata for all session logs belonging to the active project, including session_id, turn_count, req_count, project_name, started_at, updated_at, filename, and is_active flags.

ID	Category	Requirement
FR-44	Logging	The system shall expose GET /api/logs/<id>/replay returning full conversation turns, domain gate state, and NFR coverage from either live in-memory session or from the on-disk log file.
FR-45	Logging	The system shall expose GET /api/logs/<id>/download providing raw log file download by session ID.
FR-46	Logging	The system shall expose GET /api/health returning the server status, active LLM provider name, and iteration version string.

3.2 External Interface Requirements

3.2.1 User Interfaces

The RE Assistant provides a single-page web application (index.html) served by the Flask server at the root URL (/). The UI consists of the following functional areas:

- Chat Panel: A conversational interface with a message input field, send button, and scrollable message history. Messages are rendered as styled chat bubbles distinguishing user and assistant turns.
- Project Panel: A sidebar displaying the active project name, task type, and requirement count. Includes controls for creating, switching, and deleting projects.
- Domain Gate Ring: A circular progress indicator showing the percentage of functional domains confirmed (0–100%). Colour-coded: grey (< 40%), amber (40–79%), green (≥ 80%).
- IEEE-830 Coverage Ring: A circular progress indicator showing the percentage of IEEE 830 categories covered. Uses the same colour scheme as the domain gate ring.
- Gap Panel: A collapsible panel listing coverage gaps by IEEE 830 category, updated after each conversation turn.
- Session Log Browser: A panel listing all sessions associated with the current project, with replay and download controls.
- Ablation Toggle: A switch visible before session start that enables or disables gap detection for the upcoming session.

3.2.2 Software Interfaces

Interface	Protocol / Format	Description
OpenAI API	HTTPS / JSON (REST)	POST /v1/chat/completions, primary LLM backend. Model: gpt-4o. Temperature: 0.0. API key via OPENAI_API_KEY env var.
Ollama API	HTTPS / JSON (REST)	POST <OLLAMA_BASE_URL>/api/chat, alternative LLM backend hosted at University of Hildesheim. API key via OLLAMA_API_KEY env var.
Stub Provider	In-process / Python	Returns deterministic canned responses for offline testing without LLM access.
File System	Local POSIX I/O	Read/write access to logs/, output/, projects/ directories. All files are UTF-8 encoded JSON or Markdown.

3.2.3 Hardware Interfaces

No direct hardware interfaces are required. The system operates as a standard server-side web application and communicates with all external services over standard TCP/IP networking. Hardware-specific requirements (server provisioning, GPU for local LLM inference) are outside the scope of this specification.

3.2.4 Communications Interfaces

The system uses HTTP/1.1 or HTTP/2 over TCP/IP for all client-server and server-LLM communication. CORS (Cross-Origin Resource Sharing) is enabled on the Flask server via flask-cors to allow the browser UI to make API calls from any origin. All API request and response bodies use application/json content type.

3.3 Performance Requirements

ID	Category	Requirement
NFR-01	Performance	The system shall return an assistant response from the LLM and complete all post-processing steps (extraction, SMART check, domain matching, gap analysis) within 30 seconds for 95% of conversation turns under normal network conditions.
NFR-02	Performance	The system shall complete SRS document generation (finalize_session) within 120 seconds for sessions with up to 100 extracted requirements, including the SRS Coverage Enricher LLM synthesis pass.
NFR-03	Performance	The system shall complete requirements file preprocessing (upload pipeline) within 60 seconds for batches of up to 50 requirements.
NFR-04	Performance	The Flask HTTP server shall handle at least 10 concurrent REST requests without returning HTTP 5xx errors, accepting that LLM-bound requests may queue.

3.4 Logical Database Requirements

The RE Assistant does not use a relational or document database. All persistence is achieved through JSON files on the local file system. The logical data model is as follows:

- Project Record (projects/<id>.json): id (string), name (string), description (string), task_type (elicitation | srs_only), created_at (ISO 8601 timestamp), updated_at (ISO 8601 timestamp), session_id (string | null), req_count (integer), srs_path (string | null).
- Session Log (logs/session_<id>.json): A JSON array of event objects. Each event has event_type (string), timestamp (Unix epoch float), and a data payload specific to the event type. Event types include: session_start, turn, phase4_sections_stored, requirements_injected, srs_coverage_fill, session_end.
- SRS Document (output/SRS_<id>_<timestamp>.md): A Markdown-formatted plain text file. No structured data schema; human-readable narrative with embedded requirement tables.

No data migration strategy is required given the single-deployment, research-prototype nature of the system.

3.5 Design Constraints

ID	Category	Constraint
CON-01	Constraint	The system shall be implemented exclusively in Python (backend) and vanilla HTML/CSS/JavaScript (frontend), with no

ID	Category	Constraint
		frontend build toolchain (no Webpack, Babel, or transpilation step) required for deployment.
CON-02	Constraint	All LLM calls shall use a temperature setting of 0.0 to maximise output determinism and reproducibility for evaluation purposes.
CON-03	Constraint	Session state shall be stored exclusively in server memory; no database server (SQL or NoSQL) shall be required for deployment.
CON-04	Constraint	The system shall support only single-user operation; no authentication, authorisation, or multi-tenancy mechanism is required.
CON-05	Constraint	The generated SRS document shall be saved in Markdown format (.md) and shall not require any post-processing toolchain for human readability.

3.6 System Quality Attributes

Non-functional quality attributes are organised by the six mandatory NFR categories enforced by the RE Assistant itself. Each category requires at least two measurable requirements, consistent with the system's own NFR depth enforcement rule (MIN_NFR_PER_CATEGORY = 2).

3.6.1 Performance

ID	Category	Requirement
NFR-01	Performance	The system shall return an assistant response from the LLM and complete all post-processing steps (extraction, SMART check, domain matching, gap analysis) within 30 seconds for 95% of conversation turns under normal network conditions.
NFR-02	Performance	The system shall complete SRS document generation (finalize_session) within 120 seconds for sessions with up to 100 extracted requirements, including the SRS Coverage Enricher LLM synthesis pass.
NFR-03	Performance	The system shall complete requirements file preprocessing (upload pipeline) within 60 seconds for batches of up to 50 requirements.
NFR-04	Performance	The Flask HTTP server shall handle at least 10 concurrent REST requests without returning HTTP 5xx errors, accepting that LLM-bound requests may queue.

3.6.2 Usability

ID	Category	Requirement
NFR-05	Usability	The single-page web UI shall render completely and be interactive in under 3 seconds on a standard broadband

ID	Category	Requirement
		connection (>10 Mbps) without requiring any browser plugins or extensions.
NFR-06	Usability	The domain gate ring, IEEE-830 coverage ring, and gap panel shall update in real time after each conversation turn, reflecting the current session state within 500 ms of the API response being received by the browser.
NFR-07	Usability	The system shall provide inline contextual feedback in the UI for each detected coverage gap, identifying the specific IEEE 830 category that is missing or thin, without requiring the user to navigate to a separate settings or report view.
NFR-08	Usability	A new user with knowledge of requirements engineering but no prior exposure to the RE Assistant shall be able to complete a full elicitation session and generate an SRS document without consulting external documentation, as evaluated by task completion in a usability study.

3.6.3 Security / Privacy

ID	Category	Requirement
NFR-09	Security / Privacy	The system shall not transmit personally identifiable information (PII) to the LLM provider beyond session metadata (session ID, project name) and the text of requirements as provided by the user.
NFR-10	Security / Privacy	API keys (OPENAI_API_KEY, OLLAMA_API_KEY) shall be read exclusively from environment variables and shall never be hardcoded in source files, configuration files, or log output.
NFR-11	Security / Privacy	Session IDs shall be generated using Python's uuid.uuid4() function, providing at least 122 bits of entropy per session identifier.
NFR-12	Security / Privacy	The session log download endpoint (GET /api/logs/<id>/download) shall sanitise the session_id path parameter by stripping all characters except alphanumerics and hyphens, and limit the parameter to 40 characters, preventing path traversal attacks.

3.6.4 Reliability

ID	Category	Requirement
NFR-13	Reliability	The system shall handle LLM API errors (timeouts, rate limits, malformed responses) gracefully, logging the error and returning a descriptive HTTP 500 response to the client without terminating the Flask server process.
NFR-14	Reliability	The requirement extraction fallback parser shall recover at least 80% of requirements from assistant responses where the

ID	Category	Requirement
		primary REQ tag parser fails, as measured over a test set of 50 malformed response samples.
NFR-15	Reliability	All session log writes and project file writes shall be atomic at the file level (write-then-rename or equivalent), preventing partial writes from corrupting persisted data.
NFR-16	Reliability	The domain re-seeding mechanism shall execute at turns 10, 20, and 30 without raising unhandled exceptions even if the LLM returns an empty or malformed domain list; in such cases the existing domain set shall be preserved unchanged.

3.6.5 Maintainability

ID	Category	Requirement
NFR-17	Maintainability	All Python source files shall include module-level docstrings. All public classes and functions with non-trivial logic shall include docstrings describing purpose, parameters, and return values.
NFR-18	Maintainability	The LLM provider interface (LLMProvider) shall be an abstract base class with a single required method (chat), allowing a new provider to be added by implementing this interface and registering it in create_provider() without modifying any other component.
NFR-19	Maintainability	Configurable thresholds (MIN_FUNCTIONAL_REQS, MIN_NFR_PER_CATEGORY, _DOMAIN_GATE_COVERAGE_FRACTION, RESEED_TURN, MAX_HISTORY_TURNS) shall be defined as named constants in their respective modules, not as inline magic numbers.
NFR-20	Maintainability	The system architecture shall follow a component-based design where each major subsystem (ConversationManager, PromptArchitect, GapDetector, DomainDiscovery, SRSFormatter, RequirementPreprocessor) is encapsulated in its own Python module with a single, well-defined public interface.

3.6.6 Compatibility

ID	Category	Requirement
NFR-21	Compatibility	The server-side application shall run without modification on Python 3.10, 3.11, and 3.12 on Linux (Ubuntu 22.04 LTS) and macOS 13+.
NFR-22	Compatibility	The client-side single-page application shall render correctly and be fully functional in the latest stable versions of Chrome, Firefox, Safari, and Edge at the time of release.
NFR-23	Compatibility	The generated SRS Markdown output shall be renderable without modification by standard Markdown processors (GitHub

ID	Category	Requirement
		Flavored Markdown, Pandoc 2.x+) and shall produce a well-formed HTML document when converted.
NFR-24	Compatibility	The REST API shall communicate exclusively using JSON over HTTP/HTTPS and shall not depend on session cookies, browser storage, or any client-side state mechanism beyond the session_id token passed in request bodies.

Appendix A: Requirements Traceability Matrix

The following matrix traces each requirement to its functional/quality area, related requirements or categories, elicitation source, and primary implementing component.

Req ID	Category	Related To	Source	Component
FR-01	Session Mgmt	Domain Gate	Elicited	app.py / session start
FR-02	Session Mgmt	Domain Gate	Elicited	conversation_state.py
FR-03	Logging	Reliability	Elicited	session_logger.py
FR-04	Project Mgmt	Maintainability	Elicited	app.py
FR-05	Project Mgmt	Maintainability	Elicited	app.py
FR-06	Project Mgmt	Usability	Elicited	app.py
FR-07	Task Modes	Domain Gate	Elicited	conversation_manager.py
FR-08	Task Modes	Domain Gate	Elicited	prompt_architect.py
FR-09	Elicitation	Performance	Elicited	conversation_manager.py
FR-10	Elicitation	Usability	Elicited	prompt_architect.py
FR-11	Elicitation	Performance	Elicited	conversation_manager.py
FR-12	Elicitation	Reliability	Elicited	requirement_extractor.py
FR-13	Elicitation	Reliability	Elicited	requirement_extractor.py
FR-14	Logging	Reliability	Elicited	session_logger.py
FR-15	Domain Gate	Domain Gate	Elicited	domain_discovery.py
FR-16	Domain Gate	Domain Gate	Elicited	domain_discovery.py
FR-17	Domain Gate	Domain Gate	Elicited	domain_gate.py
FR-18	Domain Gate	Domain Gate	Elicited	domain_gate.py
FR-19	Domain Gate	Domain Gate	Elicited	domain_discovery.py
FR-20	Domain Gate	Domain Gate	Elicited	domain_gate.py
FR-21	Requirements	Reliability	Elicited	requirement_extractor.py
FR-22	Requirements	Usability	Elicited	conversation_manager.py
FR-23	Requirements	Domain Gate	Elicited	domain_discovery.py
FR-24	Requirements	Reliability	Elicited	conversation_manager.py
FR-25	Requirements	Maintainability	Elicited	conversation_state.py
FR-26	Decomposition	Domain Gate	Elicited	domain_discovery.py
FR-27	Decomposition	Domain Gate	Elicited	conversation_manager.py
FR-28	Decomposition	Domain Gate	Elicited	conversation_manager.py

Req ID	Category	Related To	Source	Component
FR-29	Phase Transitions	Domain Gate	Elicited	prompt_context.py
FR-30	Phase Transitions	NFR Coverage	Elicited	prompt_context.py
FR-31	Phase Transitions	Reliability	Elicited	app.py
FR-32	Gap Detection	IEEE-830	Elicited	gap_detector.py
FR-33	Gap Detection	Usability	Elicited	gap_detector.py
FR-34	Gap Detection	Usability	Elicited	prompt_context.py
FR-35	SRS Generation	IEEE-830	Elicited	srs_formatter.py
FR-36	SRS Generation	IEEE-830	Elicited	srs_formatter.py
FR-37	SRS Generation	IEEE-830	Elicited	srs_coverage.py
FR-38	SRS Generation	IEEE-830	Elicited	srs_formatter.py
FR-39	SRS Generation	IEEE-830	Elicited	srs_formatter.py
FR-40	Upload Pipeline	Usability	Elicited	app.py
FR-41	Upload Pipeline	Usability	Elicited	requirement_preprocessor.py
FR-42	Upload Pipeline	Domain Gate	Elicited	domain_gate.py
FR-43	Logging	Usability	Elicited	app.py
FR-44	Logging	Usability	Elicited	app.py
FR-45	Logging	Usability	Elicited	app.py
FR-46	Logging	Reliability	Elicited	app.py
NFR-01	Performance	FR-09	Elicited	conversation_manager.py
NFR-02	Performance	FR-35	Elicited	srs_formatter.py
NFR-03	Performance	FR-40	Elicited	requirement_preprocessor.py
NFR-04	Performance	FR-09	Elicited	app.py
NFR-05	Usability	FR-09	Elicited	index.html
NFR-06	Usability	FR-32	Elicited	index.html
NFR-07	Usability	FR-34	Elicited	index.html
NFR-08	Usability	FR-09	Elicited	index.html
NFR-09	Security/Privacy	FR-40	Elicited	llm_provider.py
NFR-10	Security/Privacy	FR-09	Elicited	llm_provider.py
NFR-11	Security/Privacy	FR-01	Elicited	conversation_manager.py
NFR-12	Security/Privacy	FR-44	Elicited	app.py

Req ID	Category	Related To	Source	Component
NFR-13	Reliability	FR-09	Elicited	conversation_manager.py
NFR-14	Reliability	FR-12	Elicited	requirement_extractor.py
NFR-15	Reliability	FR-03	Elicited	session_logger.py
NFR-16	Reliability	FR-16	Elicited	domain_discovery.py
NFR-17	Maintainability	All	Elicited	All modules
NFR-18	Maintainability	FR-09	Elicited	llm_provider.py
NFR-19	Maintainability	FR-17–31	Elicited	All modules
NFR-20	Maintainability	All	Elicited	All modules
NFR-21	Compatibility	All	Elicited	All modules
NFR-22	Compatibility	FR-05	Elicited	index.html
NFR-23	Compatibility	FR-35	Elicited	srs_formatter.py
NFR-24	Compatibility	FR-09	Elicited	app.py
CON-01	Constraint	All	Elicited	app.py / index.html
CON-02	Constraint	FR-09	Elicited	conversation_manager.py
CON-03	Constraint	FR-02	Elicited	app.py
CON-04	Constraint	FR-01	Elicited	app.py
CON-05	Constraint	FR-35	Elicited	srs_formatter.py

Appendix B: IEEE 830 Coverage and Quality Report

B.1 IEEE 830 Section Coverage

IEEE 830 Category	Evidence	Status	Coverage %
Functional Requirements	46 FRs across 12 functional categories	Covered	100%
Performance Requirements	4 measurable NFRs with quantified thresholds	Covered	100%
Usability Requirements	4 NFRs including UI response times and discoverability	Covered	100%
Security and Privacy Requirements	4 NFRs covering PII handling, key management, session entropy	Covered	100%
Reliability Requirements	4 NFRs covering error handling, fallback extraction, atomic I/O	Covered	100%
Maintainability Requirements	4 NFRs covering architecture, extensibility, constants	Covered	100%
Compatibility Requirements	4 NFRs covering Python versions, browsers, Markdown output	Covered	100%
Design Constraints	5 constraints covering stack, temperature, storage, auth, output format	Covered	100%
External Interface Requirements	UI, software interfaces (OpenAI/Ollama/File System), hardware, comms	Covered	100%
Logical Database Requirements	File-based persistence model documented	Covered	100%
Product Functions (§2.2)	12 major functions enumerated	Covered	100%
Operating Environment (§2.4)	Server, client, network, LLM, storage requirements specified	Covered	100%
User Characteristics (§2.3)	4 user classes with technical level profiles	Covered	100%
Assumptions and Dependencies (§2.5)	8 numbered assumptions specified	Covered	100%

B.2 SMART Quality Assessment

All 46 functional requirements and 29 non-functional requirements / constraints in this SRS have been evaluated against SMART criteria:

Criterion	Target	Assessment
Specific	All reqs name a single behaviour	All requirements use 'shall' with a specific system action or measurable property. No ambiguous generalities.
Measurable	NFRs include quantified thresholds	All NFRs include numeric thresholds (e.g., 30 seconds, 95%, 0.7 similarity, 80% domain coverage). Functional requirements specify observable system actions.
Achievable	Verified against architecture	All requirements are consistent with the Python + Flask + LLM architecture. No requirement assumes capabilities unavailable in the selected technology stack.
Relevant	Aligned with research goals	All requirements are traceable to the research objectives of automating RE elicitation and generating IEEE 830-compliant SRS documents.
Time-bound	Thresholds on temporal NFRs	All performance NFRs specify response time thresholds. The development schedule is governed by the University of Hildesheim project timeline.

B.3 Requirements Count Summary

Requirement Type	Count	Notes
Functional Requirements (FR)	46	Across 12 functional areas
Non-Functional Requirements (NFR)	24	6 categories × 4 requirements each
Constraints (CON)	5	Technology, quality, and deployment constraints
TOTAL	75	All requirements specified in this SRS

Appendix C: System Architecture Overview

C.1 High-Level Architecture

The RE Assistant follows a layered, component-based architecture. The browser communicates exclusively with the Flask REST API. The API layer coordinates all business logic through the ConversationManager, which orchestrates six subsystems:

Component	Module(s)	Responsibility
ConversationManager	conversation_manager.py	Orchestrates the full turn lifecycle: prompt construction, LLM call, extraction, domain matching, SMART check, decomposition, gap analysis, logging.
PromptArchitect	prompt_architect.py, prompt_context.py	Builds phase-aware system prompts for each LLM call. Selects domain context, NFR context, or IEEE section context based on current phase.
RequirementExtractor	requirement_extractor.py	Parses REQ and SECTION XML tags from assistant responses. Provides fallback regex extraction for malformed output.
DomainDiscovery + DomainGate	domain_discovery.py, domain_gate.py, domain_space.py	Seeds and tracks functional domains. Classifies requirements by domain and NFR category. Manages probe questions and decomposition.
GapDetector	gap_detector.py	Analyses requirements against 14 IEEE 830 categories. Returns per-category coverage status and gap descriptions.
SRSTemplate + SRSFormatter	srs_template.py, srs_formatter.py	Maintains the SRS data model during elicitation. Generates the final Markdown SRS document at session end.
SRSCoverageEnricher	srs_coverage.py	Fills empty SRS sections using Phase 3 answers, LLM synthesis, or architect-review stubs.
RequirementPreprocessor	requirement_preprocessor.py	Processes uploaded requirements files: SMART rewrite, type classification, category assignment, atomic splitting.
LLMProvider	llm_provider.py	Abstract interface for LLM backends. Concrete implementations: OpenAIProvider, OllamaProvider, StubProvider.
SessionLogger	session_logger.py	Writes structured JSON event logs to logs/session_<id>.json.
ConversationState	conversation_state.py	In-memory data model for all session state: turns, requirements, NFR coverage, domain gate, Phase 3 sections.

C.2 Request Flow, Elicitation Mode (Per Turn)

The following steps describe the complete processing pipeline for a single conversation turn in elicitation mode:

1. Pre-call: Probe-count increment for the last probed domain; domain gate status refresh.
2. Prompt construction: PromptArchitect.build_system_message() selects phase (fr / nfr / ieee) and injects domain context, coverage hints, or IEEE section context.
3. LLM call: LLMProvider.chat() sends the system message and conversation history (last 10 turns) to the configured backend. Temperature = 0.0.
4. Requirement extraction: RequirementExtractor parses REQ tags from the response; fallback regex runs if primary parsing yields nothing.
5. Section extraction: RequirementExtractor parses SECTION tags and commits them to ConversationState.phase4_sections_covered.
6. SMART check: LLM batch call evaluates and rewrites vague requirements before committing.
7. Domain matching: Each extracted requirement is matched to its functional domain key via DomainDiscovery.match_requirement_to_domain().
8. NFR classification: Non-functional requirements are classified into one of 6 NFR categories via DomainDiscovery.classify_nfr().
9. Sub-dimension classification: Requirements are tagged with domain sub-dimensions for coverage template cross-checking.
10. Commit: Deduplicated requirements are committed to ConversationState.requirements with full metadata.
11. Domain status update: DomainDiscovery.update_domain_statuses() re-evaluates all domain statuses after the new requirements are committed.
12. Decomposition: Template-aware decomposition runs for eligible domains (≥ 2 requirements, not yet decomposed or grown by ≥ 3 since last pass).
13. SRS template sync: SRSTemplate.update_from_requirements() refreshes the template data model.
14. Gap analysis: GapDetector.analyse() produces an updated gap report.
15. Turn logging: SessionLogger.log_turn() persists the turn to the session log file.

C.3 API Endpoint Summary

Endpoint	Method	Description
GET /api/projects	GET	List all projects (card data, sorted by modification time).
POST /api/projects/create	POST	Create a new project (name, description, task_type).
GET /api/projects/<id>	GET	Retrieve a single project by ID.
PUT /api/projects/<id>	PUT	Update project name or description.
DELETE /api/projects/<id>	DELETE	Delete a project by ID.
POST /api/session/start	POST	Start a new session (gap_detection, task_type, project_id).

Endpoint	Method	Description
POST /api/session/chat	POST	Send a user message; returns assistant response + coverage data.
POST /api/session/upload_requirements	POST	Upload and preprocess a requirements file.
POST /api/session/generate_srs	POST	Finalize the session and generate the SRS document.
GET /api/session/coverage	GET	Return current coverage and domain gate status.
GET /api/session/requirements	GET	Return all extracted requirements for the active session.
GET /api/logs	GET	List session log metadata for the active project.
GET /api/logs/<id>/replay	GET	Return conversation turns and domain/NFR snapshots for replay.
GET /api/logs/<id>/download	GET	Download raw session log JSON file.
GET /api/health	GET	Server health check (status, provider, version).

Appendix D: Known Limitations and Future Work

D.1 Known Limitations

The following limitations are acknowledged in the current implementation and should be considered by any evaluator or future maintainer:

ID	Limitation	Impact / Mitigation
L-01	Session persistence: sessions are stored in memory only	Restarting the server loses all active sessions. Mitigation: session logs and generated SRS files are persisted to disk and can be replayed or retrieved after restart.
L-02	High LLM call volume per turn	Multiple LLM calls per turn (domain matching, NFR classification, SMART check, decomposition, probe generation) cause noticeable latency on high-latency providers. Mitigation: temperature=0.0 and MAX_HISTORY_TURNS=10 reduce token volume.
L-03	Domain gate seeding quality depends on first message richness	Vague opening messages yield generic or incomplete domain lists. Mitigation: re-seeding at turns 10, 20, and 30 progressively refines the domain set.
L-04	Probe-count dependency in srs_only mode	Domains seeded from uploaded requirement labels remain in partial status until actively probed, potentially slowing Phase 1 advancement in srs_only mode.
L-05	SMART heuristics for decomposed requirements	Decomposed requirements rely on lightweight keyword/regex SMART heuristics rather than the LLM-based batch check applied to elicited requirements.
L-06	Requirement extraction reliability	Extraction depends on the LLM consistently emitting well-formed REQ and SECTION tags. Malformed or missing tags trigger fallback patterns that may miss requirements.
L-07	High-risk section stubs require manual completion	Hardware interfaces, logical database requirements, and design constraints sections are always stubbed and require manual completion by a system architect before the SRS is usable for development.
L-08	Single-user, no authentication	Not intended for multi-user or production deployment. Concurrent access to the same session from multiple browser tabs may cause state inconsistencies.

D.2 Future Work

The following enhancements are identified as candidates for future development iterations beyond Iteration 8:

- Persistent Session Storage: Serialize ConversationState to disk (e.g., SQLite or Redis) to survive server restarts.
- Multi-User Support: Introduce user authentication (OAuth 2.0 or session-based) and per-user project namespacing.
- Interactive SRS Editor: Allow stakeholders to review, annotate, and approve individual requirements within the UI before generating the final SRS.
- Integration with Requirement Management Tools: Export generated SRS requirements to Jira, Azure DevOps, or IBM DOORS via REST API connectors.
- Evaluation Dashboard: Automated scoring of elicitation quality (SMART score distribution, domain coverage, NFR depth) with comparison across ablation study conditions.
- Support for Multiple LLM Providers Simultaneously: Route different subtasks (e.g., domain seeding vs. SMART rewriting) to different models optimized for those tasks.
- Conversational Follow-up Clarification: Allow the RE Assistant to ask clarifying follow-up questions when a requirement is ambiguous, rather than silently flagging it as ambiguous.
- Diagram Generation: Automatically generate use case diagrams or domain model sketches from elicited requirements using a diagram generation LLM.